



*Department of Computer Science and Engineering (CSE)
Semester: (Summer, Year: 2025), B.Sc. in CSE (Day)*

Alert & Notification System

*Course Title: Computer Networking Lab
Course Code: CSE 312
Section: 231 D2*

Students Details

Name	ID
MD MAHABUB HASAN MAHIN	231902056
MAJAHARUL ISLAM	231902050

*Submission Date: 26-08-2025
Course Teacher's Name: Fatema Akter*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	2
1.1	Overview	2
1.2	Motivation	2
1.2.1	Problem Statement	2
1.3	Design Goals/Objectives	3
1.4	Technologies Used (Database and Java UI Design)	3
1.5	Features	3
2	Implementation of the Project	5
2.1	Introduction	5
2.2	Design	5
2.3	Implementation of the Alert & Notification System	7
3	Performance Evaluation	18
3.1	Results Analysis	18
4	Conclusion	23
4.1	Discussion	23
4.2	Limitations	23
4.3	Scope of Future Work	23

Chapter 1

Introduction

1.1 Overview

The Alert & Notification System is a client-server messaging application designed to manage alerts and notifications. Multiple clients can connect to a central server, send messages and receive replies from the server. The server organizes messages as alerts, allowing users to mark them as read or unread and reply directly to clients. The system ensures efficient real-time communication and provides a user-friendly GUI for both clients and the server.

1.2 Motivation

Modern organizations, laboratories and work environments require instant notifications and communication among multiple users or departments. Manual alert handling can be inefficient and prone to delays, which may result in missed important messages. This project aims to automate the alert notification system to enable faster, more reliable and structured communication.

1.2.1 Problem Statement

In a multi-user environment, communication delays or missed notifications can lead to inefficiencies. There is a need for a system that allows multiple users to send messages, ensures alerts are visible to the administrator or server and provides a way to mark messages as read or unread and reply directly—all through a simple graphical interface.

1.3 Design Goals/Objectives

- Enable multiple clients to connect and communicate with the server in real time.
- Provide a system to track and manage alert messages efficiently.
- Notify the server of incoming messages with optional auditory or pop-up alerts.
- Allow the server to reply to clients.
- Maintain the read/unread status of messages.
- Ensure a user-friendly interface with clear GUI components.

1.4 Technologies Used (Database and Java UI Design)

1. Programming Language & UI :

- Java SE
- Java Swing for GUI design

2. Database/Persistent Storage :

- File handling (client_counter.txt) for storing client numbers across sessions

3. Networking:

- Socket programming (Socket, ServerSocket) for client-server communication
- Multi-threading using ExecutorService for handling multiple clients

1.5 Features

Client Features:

- Connect to and disconnect from the server
- Auto-generated unique client name
- Send messages to the server
- View server replies in message history and pop-up notifications

Server Features :

- Start and stop the server on a specified port
- Display client messages as alerts with details and timestamps
- Mark messages as read or unread
- Display total and unread message counts
- Auditory and pop-up notifications for new messages
- Reply directly to clients

Chapter 2

Implementation of the Project

2.1 Introduction

The project is implemented in Java using Swing for the user interface and socket programming for real-time network communication. Multi-threading is used on the server side to manage multiple client connections simultaneously.

2.2 Design

Client Design:

- GUI components include host, port, and client name fields; a message input area; a send button; and a message history list.
- A separate thread reads server replies and displays them in a message list and temporary pop-up.

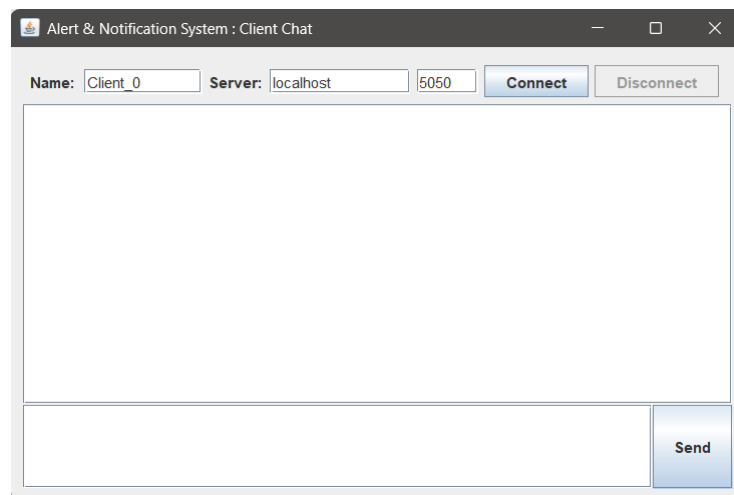


Figure 2.1: Client Design Diagram

Server Design:

- GUI components include a port field, server status label, alert list with a details area, buttons for marking messages, options for auditory/pop-up alerts, and a reply feature.
- The server uses `ExecutorService` to manage multiple clients. Each client is handled in a `ClientHandler` thread.

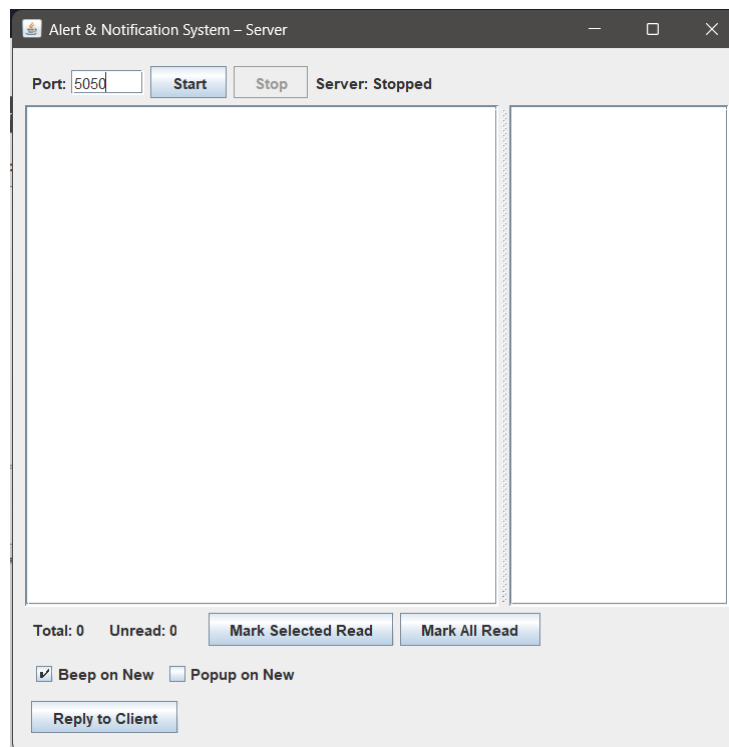


Figure 2.2: Server Design Diagram

2.3 Implementation of the Alert & Notification System

Client Code :

```
1  package com.mycompany.notification;
2
3  import javax.swing.*;
4  import javax.swing.border.EmptyBorder;
5  import java.awt.*;
6  import java.io.*;
7  import java.net.*;
8
9  public class ClientApp {
10     public static void main(String[] args) {
11         SwingUtilities.invokeLater(() -> new ClientFrame().
12             setVisible(true));
13     }
14 }
15
16 class ClientFrame extends JFrame {
17     private JTextField txtServerHost = new JTextField("localhost", 12);
18     private JTextField txtServerPort = new JTextField("5050", 5);
19     private JTextField txtName = new JTextField(10);
20     private JButton btnConnect = new JButton("Connect");
21     private JButton btnDisconnect = new JButton("Disconnect");
22
23     private DefaultListModel<String> msgModel = new
24         DefaultListModel<>();
25     private JList<String> msgList = new JList<>(msgModel);
26     private JTextArea txtMessage = new JTextArea(4, 30);
27     private JButton btnSend = new JButton("Send");
28
29     private Socket socket;
30     private BufferedReader reader;
31     private PrintWriter writer;
32     private volatile boolean connected = false;
33
34     private static final String NAME_FILE = "client_counter.txt";
35
36     private int countSentMsg = 0;
37
38     ClientFrame() {
39         super("Alert & Notification System : Client Chat");
40         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41         setSize(600, 400);
42         setLocationRelativeTo(null);
43
44         txtName.setText("Client" + getNextClientNumber());
45
46         JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
47         top.add(new JLabel("Name:"));
48         top.add(txtName);
49         top.add(new JLabel("Server:"));
50         top.add(txtServerHost);
```

```

48     top.add(txtServerPort);
49     top.add(btnConnect);
50     top.add(btnDisconnect);
51     btnDisconnect.setEnabled(false);
52
53     msgList.setSelectionMode(ListSelectionModel.
54         SINGLE_SELECTION);
55     JScrollPane scroll = new JScrollPane(msgList);
56
57     JPanel bottom = new JPanel(new BorderLayout());
58     txtMessage.setLineWrap(true);
59     txtMessage.setWrapStyleWord(true);
60     bottom.add(new JScrollPane(txtMessage), BorderLayout.
61         CENTER);
62     bottom.add(btnSend, BorderLayout.EAST);
63
64     JPanel root = new JPanel(new BorderLayout());
65     root.setBorder(new EmptyBorder(10, 10, 10, 10));
66     root.add(top, BorderLayout.NORTH);
67     root.add(scroll, BorderLayout.CENTER);
68     root.add(bottom, BorderLayout.SOUTH);
69     setContentPane(root);
70
71     btnConnect.addActionListener(e -> connect());
72     btnDisconnect.addActionListener(e -> disconnect());
73     btnSend.addActionListener(e -> sendMessage());
74 }
75
76 private int getNextClientNumber() {
77     int number = 1;
78     try {
79         File file = new File(NAME_FILE);
80         if (file.exists()) {
81             BufferedReader br = new BufferedReader(new
82                 FileReader(file));
83             String line = br.readLine();
84             if (line != null) {
85                 try { number = Integer.parseInt(line.trim())
86                     + 1; }
87                 catch (NumberFormatException ignored) {
88                     number = 1; }
89             }
90             br.close();
91         }
92         PrintWriter pw = new PrintWriter(new FileWriter(file
93             ));
94         pw.println(number);
95         pw.close();
96     } catch (IOException e) { e.printStackTrace(); }
97     return number;
98 }
99
100 private void connect() {
101     String host = txtServerHost.getText().trim();
102     int port;
103     try { port = Integer.parseInt(txtServerPort.getText().
104         trim()); }
105     catch (NumberFormatException e) { JOptionPane.

```

```

99         showMessageDialog(this, "Invalid port"); return; }
100     try {
101         socket = new Socket(host, port);
102         reader = new BufferedReader(new InputStreamReader(
103             socket.getInputStream(), "UTF-8"));
104         writer = new PrintWriter(socket.getOutputStream(),
105             true);
106
107         writer.println(txtName.getText().trim());
108         connected = true;
109
110         new Thread(this::readMessages).start();
111
112         btnConnect.setEnabled(false);
113         btnDisconnect.setEnabled(true);
114         txtName.setEnabled(false);
115         JOptionPane.showMessageDialog(this, "Connected to
116             server");
117     } catch (IOException ex) {
118         JOptionPane.showMessageDialog(this, "Connection
119             failed: " + ex.getMessage());
120     }
121
122     private void disconnect() {
123         connected = false;
124         try { if (socket != null) socket.close(); } catch (
125             IOException ignored) {}
126         btnConnect.setEnabled(true);
127         btnDisconnect.setEnabled(false);
128         txtName.setEnabled(true);
129         JOptionPane.showMessageDialog(this, "Disconnected from
130             server");
131     }
132
133     private void sendMessage() {
134         if (!connected) return;
135         String msg = txtMessage.getText().trim();
136         if (!msg.isEmpty()) {
137             String fullMsg = txtName.getText().trim() + "\t" +
138                 msg;
139             writer.println(fullMsg);
140             countSentMsg++;
141             msgModel.addElement("Sent Msg      " + msg);
142             txtMessage.setText("");
143         }
144     }
145
146     private void readMessages() {
147         try {
148             String line;
149             while (connected && reader != null && (line = reader
150                 .readLine()) != null) {
151                 final String msg = line;
152                 SwingUtilities.invokeLater(() -> {
153                     msgModel.addElement("Server: " + msg);

```

```

148         showTemporaryPopup("Server Reply:\n" + msg);
149     });
150 }
151 } catch (IOException e) {
152     if (connected) {
153         SwingUtilities.invokeLater(() ->
154             JOptionPane.showMessageDialog(this, "
155                 Connection lost: " + e.getMessage()));
156         disconnect();
157     }
158 }
159
160 private void showTemporaryPopup(String msg) {
161     JDialog dialog = new JDialog(this, "Server Reply", false
162         );
163     dialog.setSize(300, 150);
164     dialog.setLocationRelativeTo(this);
165     dialog.setLayout(new BorderLayout());
166     JLabel label = new JLabel("<html><body style='padding:10
167         px;'>" + msg + "</body></html>");
168     label.setHorizontalAlignment(SwingConstants.CENTER);
169     dialog.add(label, BorderLayout.CENTER);
170     dialog.setAlwaysOnTop(true);
171     dialog.setVisible(true);
172
173     // Auto-close after 3 seconds
174     new Timer(3000, e -> dialog.dispose()).start();
175 }

```

Listing 2.1: Client Code

Server Code :

```
1 package com.mycompany.notification;
2
3 import javax.swing.*;
4 import javax.swing.border.EmptyBorder;
5 import java.awt.*;
6 import java.io.*;
7 import java.net.*;
8 import java.text.SimpleDateFormat;
9 import java.util.*;
10 import java.util.concurrent.*;
11
12 public class ServerApp {
13     public static void main(String[] args) {
14         SwingUtilities.invokeLater(() -> new ServerFrame().
15             setVisible(true));
16     }
17
18     class ServerFrame extends JFrame {
19         private static final int DEFAULT_PORT = 5050;
20
21         private final JLabel statusLabel = new JLabel("Server:
22             Stopped");
23         private final JTextField portField = new JTextField(String.
24             valueOf(DEFAULT_PORT), 6);
25         private final JButton startBtn = new JButton("Start");
26         private final JButton stopBtn = new JButton("Stop");
27
28         private final DefaultListModel<AlertItem> alertModel = new
29             DefaultListModel<>();
30         private final JList<AlertItem> alertList = new JList<>(
31             alertModel);
32         private final JTextArea detailsArea = new JTextArea();
33
34         private final JLabel totalCountLabel = new JLabel("Total: 0"
35             );
36         private final JLabel unreadCountLabel = new JLabel("Unread:
37             0");
38         private final JButton markReadBtn = new JButton("Mark
39             Selected Read");
40         private final JButton markAllReadBtn = new JButton("Mark All
41             Read");
42         private final JCheckBox beepCheck = new JCheckBox("Beep on
43             New", true);
44         private final JCheckBox popupCheck = new JCheckBox("Popup on
45             New", false);
46         private final JButton replyBtn = new JButton("Reply to
47             Client");
48
49         private volatile ServerSocket serverSocket;
50         private ExecutorService pool;
51         private volatile boolean running = false;
52
53         // Map to store clients with unique key: host:port
54         private final Map<String, PrintWriter> clientMap =
```

```

44     Collections.synchronizedMap(new HashMap<>());
45 private final SimpleDateFormat sdf = new SimpleDateFormat("
46     yyyy-MM-dd HH:mm:ss");
47
48 ServerFrame() {
49     super("Alert & Notification System      Server");
50     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51     setSize(1000, 600);
52     setLocationRelativeTo(null);
53
54     // Top panel
55     JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT))
56         ;
57     top.add(new JLabel("Port:"));
58     top.add(portField);
59     top.add(startBtn);
60     top.add(stopBtn);
61     top.add(statusLabel);
62
63     // Alert list + details split pane
64     alertList.setSelectionMode(ListSelectionModel.
65         SINGLE_SELECTION);
66     alertList.setCellRenderer(new AlertCellRenderer());
67     JScrollPane listScroll = new JScrollPane(alertList);
68
69     detailsArea.setEditable(false);
70     detailsArea.setLineWrap(true);
71     detailsArea.setWrapStyleWord(true);
72     detailsArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN
73         , 13));
74     JScrollPane detailsScroll = new JScrollPane(detailsArea)
75         ;
76
77     JSplitPane split = new JSplitPane(JSplitPane.
78         HORIZONTAL_SPLIT, listScroll, detailsScroll);
79     split.setDividerLocation(500);
80
81     // Bottom panel
82     JPanel bottom = new JPanel();
83     bottom.setLayout(new BoxLayout(bottom, BoxLayout.Y_AXIS)
84         );
85
86     JPanel row1 = new JPanel(new FlowLayout(FlowLayout.LEFT)
87         );
88     row1.add(totalCountLabel);
89     row1.add(Box.createHorizontalStrut(10));
90     row1.add(unreadCountLabel);
91     row1.add(Box.createHorizontalStrut(15));
92     row1.add(markReadBtn);
93     row1.add(markAllReadBtn);
94     bottom.add(row1);
95
96     JPanel row2 = new JPanel(new FlowLayout(FlowLayout.LEFT)
97         );
98     row2.add(beepCheck);
99     row2.add(popupCheck);
100    bottom.add(row2);

```

```

92     JPanel row3 = new JPanel(new FlowLayout(FlowLayout.LEFT)
93         );
94     row3.add(replyBtn);
95     bottom.add(row3);
96
97     // Root layout
98     JPanel root = new JPanel(new BorderLayout());
99     root.setBorder(new EmptyBorder(10, 10, 10, 10));
100    root.add(top, BorderLayout.NORTH);
101    root.add(split, BorderLayout.CENTER);
102    root.add(bottom, BorderLayout.SOUTH);
103    setContentPane(root);
104
105    // Actions
106    startBtn.addActionListener(e -> startServer());
107    stopBtn.addActionListener(e -> stopServer());
108    markReadBtn.addActionListener(e -> markSelectedRead());
109    markAllReadBtn.addActionListener(e -> markAllRead());
110
111    alertList.addListSelectionListener(e -> {
112        AlertItem item = alertList.getSelectedValue();
113        detailsArea.setText(item != null ? renderDetails(
114            item) : "");
115    });
116
117    replyBtn.addActionListener(e -> openReplyDialog());
118    stopBtn.setEnabled(false);
119
120    }
121
122    private String renderDetails(AlertItem a) {
123        return "Time: " + sdf.format(new Date(a.timestamp)) + "\n"
124            + "From: " + a.sender + "\n"
125            + "ClientKey: " + a.clientKey + "\n"
126            + "Read: " + (a.read ? "Yes" : "No") + "\n\n"
127            + "Message:\n" + a.message;
128    }
129
130    private void startServer() {
131        if (running) return;
132        int port;
133        try {
134            port = Integer.parseInt(portField.getText().trim());
135            if (port < 1 || port > 65535) throw new
136                NumberFormatException();
137        } catch (NumberFormatException ex) {
138            JOptionPane.showMessageDialog(this, "Invalid port.");
139        }
140        return;
141    }
142
143    try {
144        serverSocket = new ServerSocket(port);
145        running = true;
146        pool = Executors.newCachedThreadPool();
147        pool.execute(this::acceptLoop);
148        statusLabel.setText("Server: Running on port " +
149            port);
150        startBtn.setEnabled(false);

```

```

144         stopBtn.setEnabled(true);
145         portField.setEnabled(false);
146     } catch (IOException ex) {
147         JOptionPane.showMessageDialog(this, "Failed to start
148             : " + ex.getMessage());
149     }
150 }
151
152 private void stopServer() {
153     running = false;
154     try { if (serverSocket != null) serverSocket.close(); }
155     catch (IOException ignored) {}
156     if (pool != null) pool.shutdownNow();
157     statusLabel.setText("Server: Stopped");
158     startBtn.setEnabled(true);
159     stopBtn.setEnabled(false);
160     portField.setEnabled(true);
161 }
162
163 private void acceptLoop() {
164     while (running) {
165         try {
166             Socket sock = serverSocket.accept();
167             pool.execute(new ClientHandler(sock, this));
168         } catch (IOException e) {
169             if (running) SwingUtilities.invokeLater(() ->
170                 JOptionPane.showMessageDialog(this, "
171                     Accept error: " + e.getMessage()));
172         }
173     }
174 }
175
176 void onNewAlert(AlertItem item) {
177     SwingUtilities.invokeLater(() -> {
178         alertModel.addElement(item);
179         updateCounts();
180         if (beepCheck.isSelected()) Toolkit.
181             getDefaultToolkit().beep();
182         if (popupCheck.isSelected()) {
183             JOptionPane.showMessageDialog(this,
184                 "From: " + item.sender + "\n" + item.
185                 message,
186                 "New Alert",
187                 JOptionPane.INFORMATION_MESSAGE);
188         }
189     });
190 }
191
192 void updateCounts() {
193     int total = alertModel.size();
194     int unread = 0;
195     for (int i = 0; i < total; i++) if (!alertModel.get(i).
196         read) unread++;
197     totalCountLabel.setText("Total: " + total);
198     unreadCountLabel.setText("Unread: " + unread);
199     alertList.repaint();
200 }

```

```

196 private void markSelectedRead() {
197     AlertItem sel = alertList.getSelectedValue();
198     if (sel != null && !sel.read) {
199         sel.read = true;
200         updateCounts();
201         detailsArea.setText(renderDetails(sel));
202     }
203 }
204
205 private void markAllRead() {
206     for (int i = 0; i < alertModel.size(); i++) alertModel.
207         get(i).read = true;
208     updateCounts();
209 }
210
211 void addClient(String clientKey, PrintWriter writer) {
212     clientMap.put(clientKey, writer); }
213
214 void removeClient(String clientKey) { clientMap.remove(
215     clientKey); }
216
217 private void openReplyDialog() {
218     AlertItem selected = alertList.getSelectedValue();
219     if (selected == null) {
220         JOptionPane.showMessageDialog(this, "Select a client
221             alert first.");
222         return;
223     }
224
225     JDialog dialog = new JDialog(this, "Reply to " +
226         selected.sender, true);
227     dialog.setSize(400, 200);
228     dialog.setLocationRelativeTo(this);
229     dialog.setLayout(new BorderLayout());
230
231     JTextArea replyArea = new JTextArea();
232     replyArea.setLineWrap(true);
233     replyArea.setWrapStyleWord(true);
234     dialog.add(new JScrollPane(replyArea), BorderLayout.
235         CENTER);
236
237     JButton sendBtn = new JButton("Send");
238     dialog.add(sendBtn, BorderLayout.SOUTH);
239
240     sendBtn.addActionListener(ev -> {
241         String msg = replyArea.getText().trim();
242         if (!msg.isEmpty()) {
243             PrintWriter out = clientMap.get(selected.
244                 clientKey);
245             if (out != null) {
246                 out.println("Server Reply: " + msg);
247
248                 AlertItem replyItem = new AlertItem(System.
249                     currentMillis(),
250                     "Server (reply to " + selected.
251                         sender + ")", msg, selected.
252                             clientKey);
253                 onNewAlert(replyItem);
254                 dialog.dispose();

```

```

244         } else {
245             JOptionPane.showMessageDialog(dialog, "
Client not available.");
246         }
247     }
248 });
249
250 dialog.setVisible(true);
251 }
252 }
253
254 class ClientHandler implements Runnable {
255     private final Socket socket;
256     private final ServerFrame server;
257     private String clientKey;
258
259     ClientHandler(Socket socket, ServerFrame server) {
260         this.socket = socket;
261         this.server = server;
262     }
263
264     @Override
265     public void run() {
266         try (BufferedReader br = new BufferedReader(new
InputStreamReader(socket.getInputStream(), "UTF-8"));
267             PrintWriter out = new PrintWriter(socket.
getOutputStream(), true)) {
268
269             // First message from client is its name
270             String clientName = br.readLine();
271             clientKey = socket.getInetAddress().getHostAddress()
+ ":" + socket.getPort();
272             server.addClient(clientKey, out);
273
274             String line;
275             while ((line = br.readLine()) != null) {
276                 String msg = line;
277                 // If client still sends name in message, remove
prefix
278                 int tab = line.indexOf('\t');
279                 if (tab >= 0) {
280                     msg = line.substring(tab + 1).trim();
281                 }
282                 AlertItem item = new AlertItem(System.
currentTimeMillis(), clientName, msg,
clientKey);
283                 server.onNewAlert(item);
284             }
285         } catch (IOException ignored) {}
286         finally {
287             try { socket.close(); } catch (IOException ignored)
{}
288             server.removeClient(clientKey);
289         }
290     }
291 }
292
293 class AlertItem {

```

```

294     final long timestamp;
295     final String sender;
296     final String message;
297     final String clientKey;
298     boolean read = false;
299
300     AlertItem(long timestamp, String sender, String message,
301               String clientKey) {
302         this.timestamp = timestamp;
303         this.sender = (sender == null || sender.isEmpty()) ? "
304             Unknown" : sender;
305         this.message = message == null ? "" : message;
306         this.clientKey = clientKey == null ? "" : clientKey;
307     }
308
309     @Override
310     public String toString() {
311         String prefix = read ? "    " : "    ";
312         String displayMsg = message.length() > 50 ? message.
313             substring(0, 50) + "... " : message;
314         return prefix + "[" + sender + "]" + displayMsg;
315     }
316 }
317
318 class AlertCellRenderer extends DefaultListCellRenderer {
319     @Override
320     public Component getListCellRendererComponent(JList<?> list,
321           Object value,
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

Listing 2.2: Server Code

Chapter 3

Performance Evaluation

3.1 Results Analysis

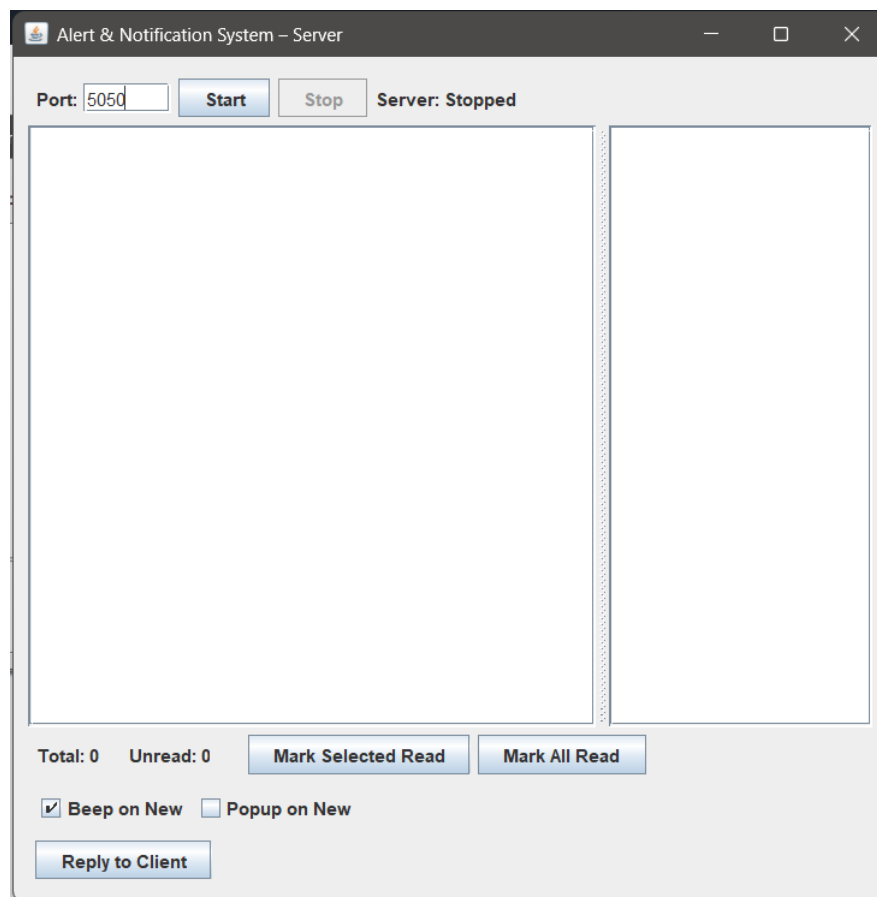


Figure 3.1: Server image without Start

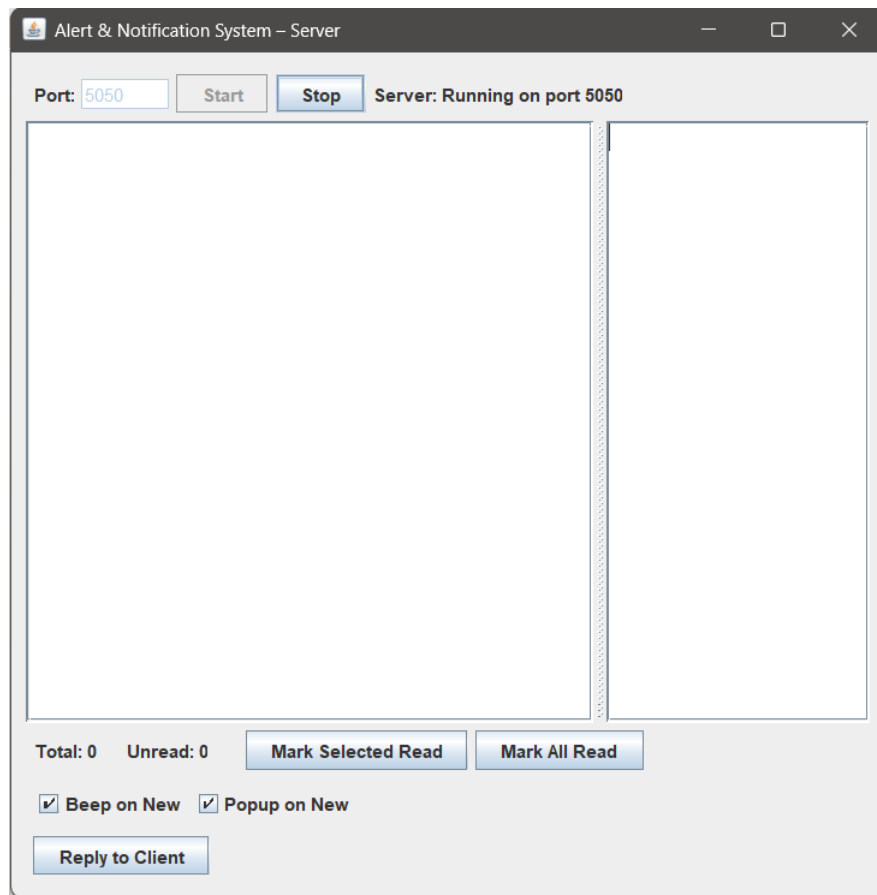


Figure 3.2: Server image Start

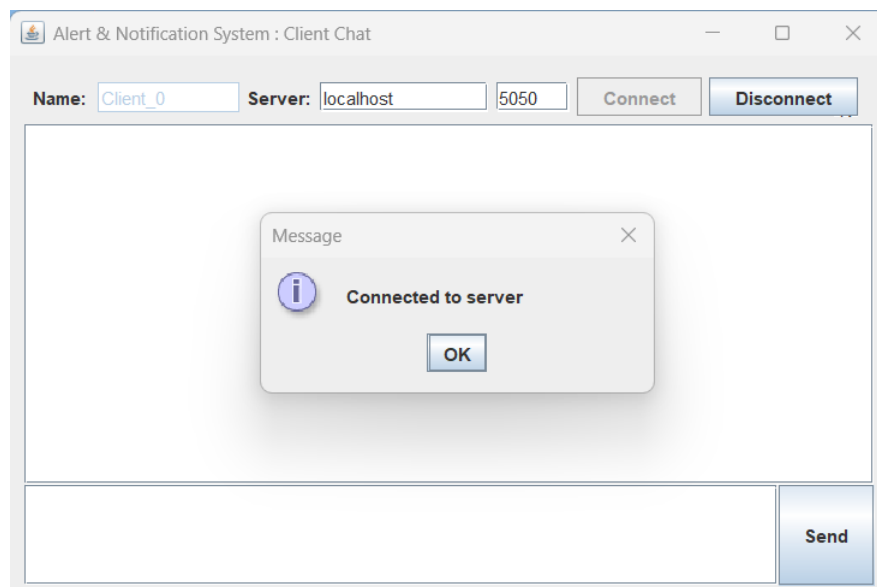


Figure 3.3: Client image Connected to Server

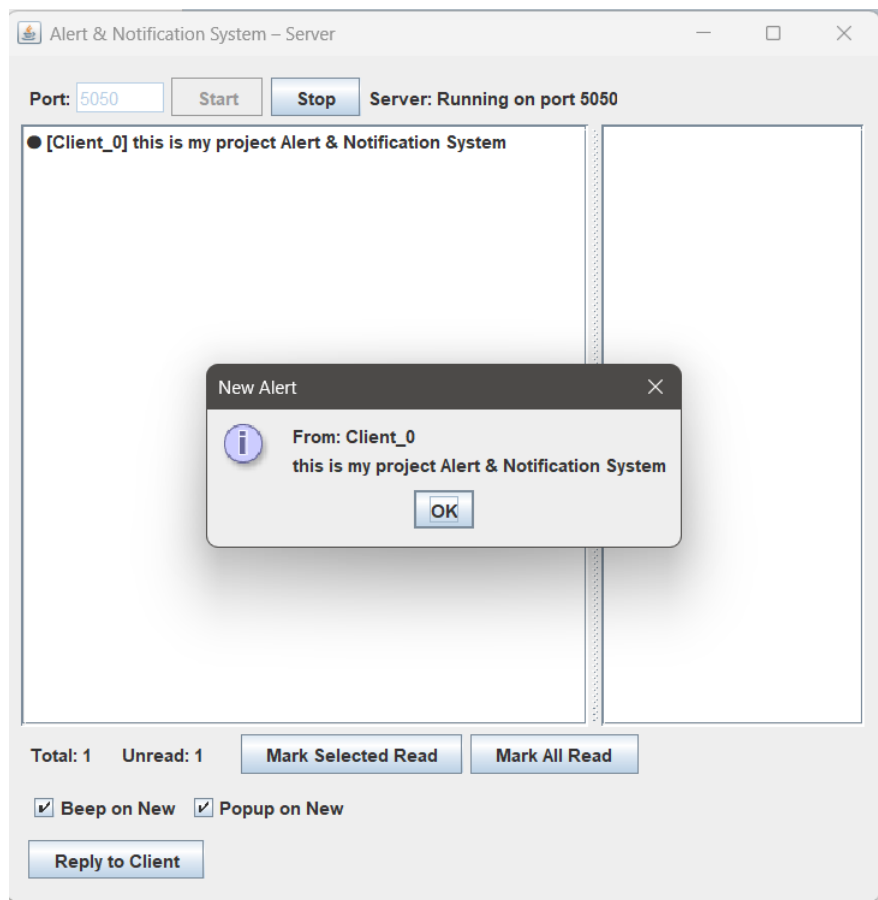


Figure 3.4: Server Alert Receive Message Client

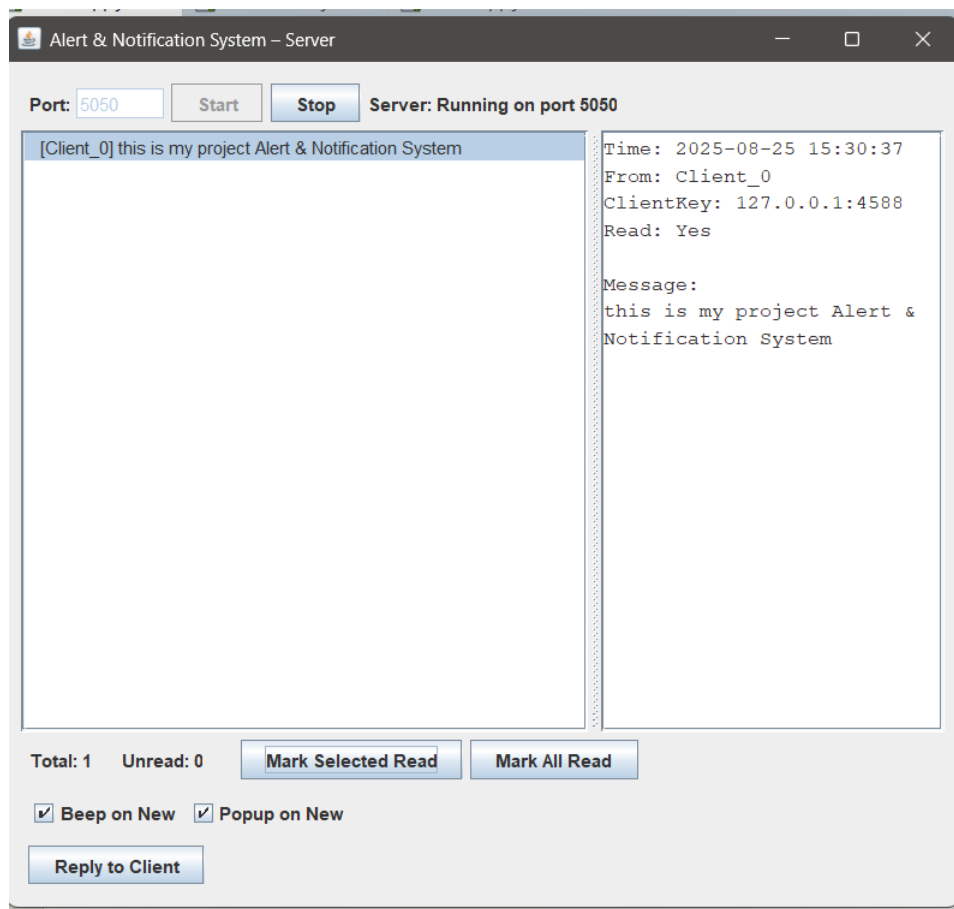


Figure 3.5: Server Read client Message

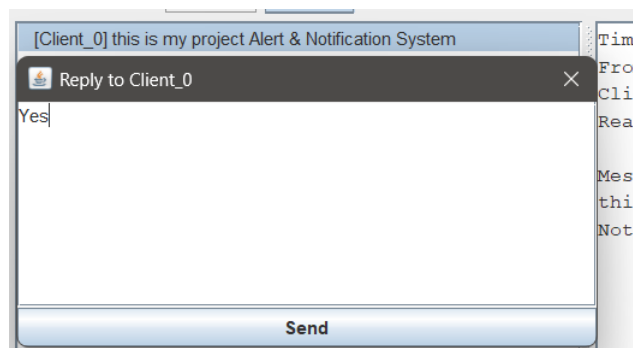


Figure 3.6: Server Reply sent to client

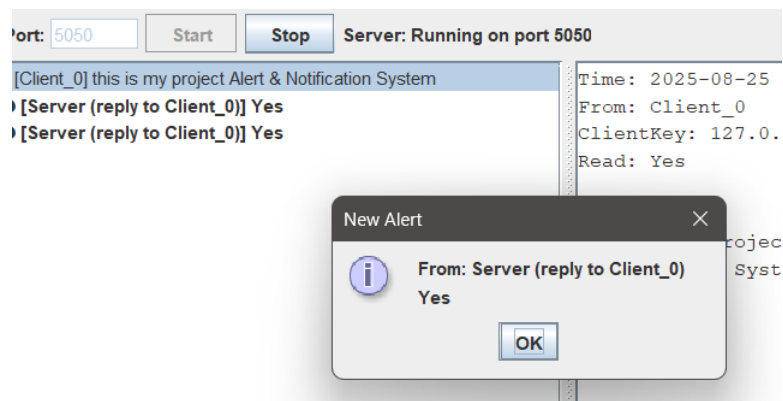


Figure 3.7: Server Reply sent complete Alert

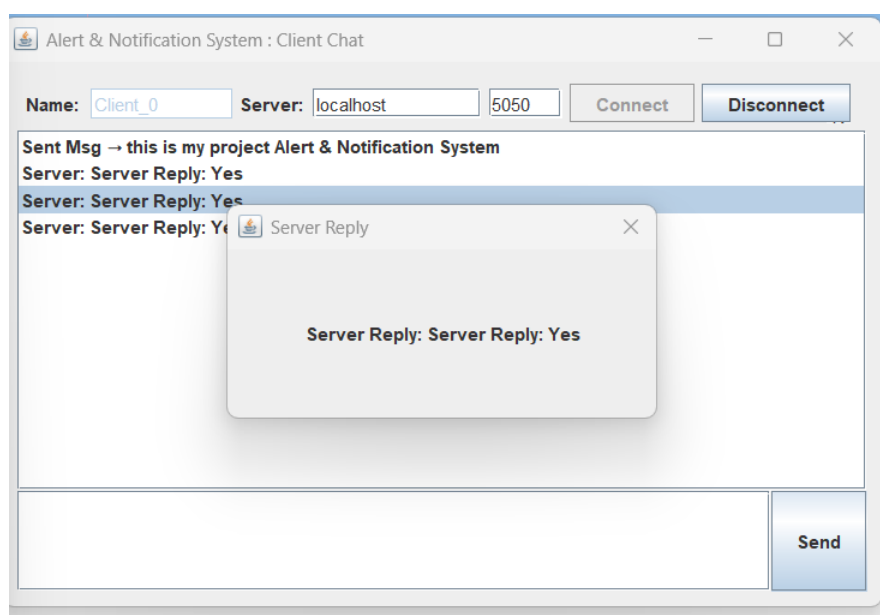


Figure 3.8: Client Receives Server Reply Message

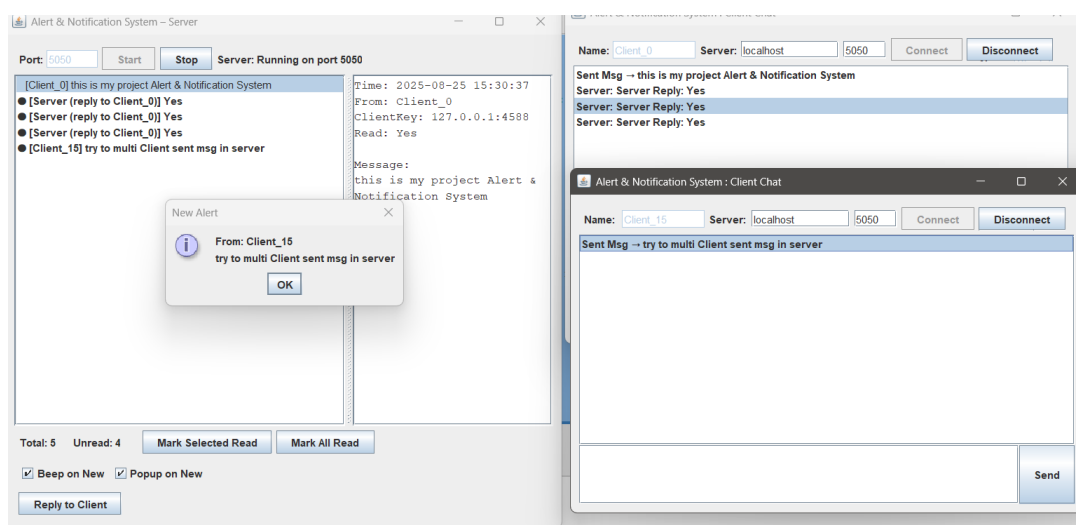


Figure 3.9: Server handle multi client message

Chapter 4

Conclusion

4.1 Discussion

The Alert & Notification System successfully provides a multi-client messaging platform with real-time alert notifications. It demonstrates the effective use of Java Swing for the GUI, socket programming for networking, and multi-threading for client handling.

4.2 Limitations

- No database integration (messages are lost on server restart).
- Messages are not encrypted; the system is not suitable for sensitive communication.
- Limited error handling for abrupt client disconnections.
- The GUI is basic and can be improved for better usability.

4.3 Scope of Future Work

- Integrate a database (e.g., MySQL) to store chat history.
- Add client authentication and role-based access.
- Encrypt messages for secure communication.
- Upgrade the GUI using JavaFX or a web interface for better user experience.
- Add advanced notification management (e.g., email or SMS alerts).